

VULNERABILITY ASSESSMENT REPORT

Target Website:
<https://demo.testfire.net>



Prepared By:
Praneetha

Tools Used:
Nmap
OWASP ZAP
Browser Developer Tools

EXECUTIVE SUMMARY

A Vulnerability Assessment was conducted on the target web application (<https://demo.testfire.net>) using industry-standard security tools including Nmap, OWASP ZAP, and Browser Developer Tools.

The objective of this assessment was to identify common web application security vulnerabilities, evaluate their severity levels, and provide appropriate remediation recommendations.

The assessment identified multiple High and Medium risk vulnerabilities, including Cross-Site Scripting (XSS), SQL Injection, Missing Anti-CSRF Protection, Missing Content Security Policy (CSP), and Missing Anti-Clickjacking Protection.

The findings and recommendations provided in this report can help improve the overall security posture of the application and reduce potential attack vectors.

ASSESSMENT METHODOLOGY

1. Target Identification

- The target web application (<https://demo.testfire.net>) was selected for security assessment.

2. Network Reconnaissance

- Nmap was used to identify open ports, services, and server information.

3. Vulnerability Assessment

- OWASP ZAP was used to identify web application vulnerabilities through automated scanning and passive analysis.

4. Manual Verification

- Browser Developer Tools were used to inspect security-related configurations and application behavior.

5. Risk Classification

- Findings were categorized as High or Medium based on their potential impact and exploitability.

NMAP SCAN RESULTS

Nmap was used to perform service detection on the target host. The scan identified several open and filtered ports along with service information running on the server.

```
C:\Users\prani>nmap -sV scanme.nmap.org
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-07 12:28 +0530
Stats: 0:00:12 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 75.00% done; ETC: 12:28 (0:00:02 remaining)
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.29s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 989 closed tcp ports (reset)
PORT      STATE      SERVICE      VERSION
22/tcp    open      ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open      http         Apache httpd 2.4.7 ((Ubuntu))
135/tcp   filtered  msrpc
139/tcp   filtered  netbios-ssn
445/tcp   filtered  microsoft-ds
1022/tcp  filtered  exp2
1023/tcp  filtered  netvenuechat
1026/tcp  filtered  LSA-or-nterm
9898/tcp  filtered  monkeycom
9929/tcp  open      nping-echo   Nping echo
31337/tcp open      tcpwrapped
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.38 seconds
```

Observation:

The scan confirmed that the target server exposes multiple services over the network. Service version information was successfully identified, which may assist attackers in reconnaissance activities if not properly secured. Risk level is low

FINDING #1

CROSS-SITE SCRIPTING(XSS)

Risk level

 High Risk

Description

Cross-Site Scripting (XSS) is a vulnerability that allows attackers to inject malicious scripts into web pages viewed by users. These scripts execute in the victim's browser and may lead to session hijacking, data theft, or unauthorized actions.

Impact

- Execution of malicious JavaScript in user browsers
- Theft of session cookies and sensitive information
- Unauthorized actions performed on behalf of users

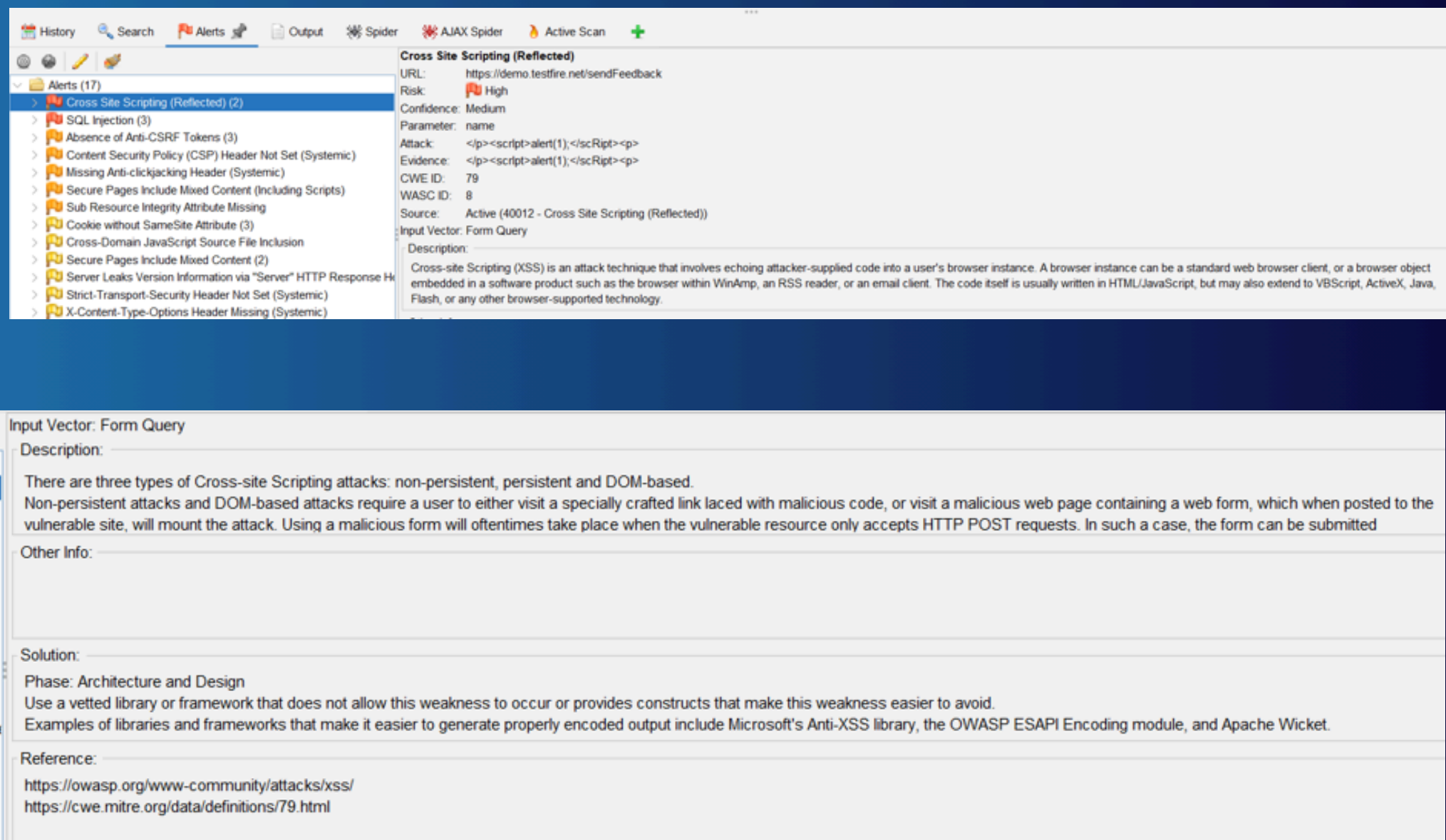
Evidence

OWASP ZAP detected a reflected Cross-Site Scripting vulnerability in the sendFeedback functionality of the application.

Remediation

- Validate and sanitize user input
- Encode output before rendering in browsers
 - Implement Content Security Policy (CSP)
 - Follow OWASP XSS Prevention guidelines

Screenshot



FINDING #2

SQL INJECTION

Risk level

 High Risk

Description

SQL Injection is a critical web application vulnerability that allows attackers to manipulate database queries by injecting malicious SQL statements through user input fields. Successful exploitation may lead to unauthorized access to sensitive information, authentication bypass, or database compromise.

Impact

- Unauthorized access to sensitive data
 - Authentication bypass
- Modification or deletion of database records
 - Complete database compromise

Evidence

OWASP ZAP identified a potential SQL Injection vulnerability in the login functionality of the application

Affected URL:

<https://demo.testfire.net/doLogin>

Parameter:

uid

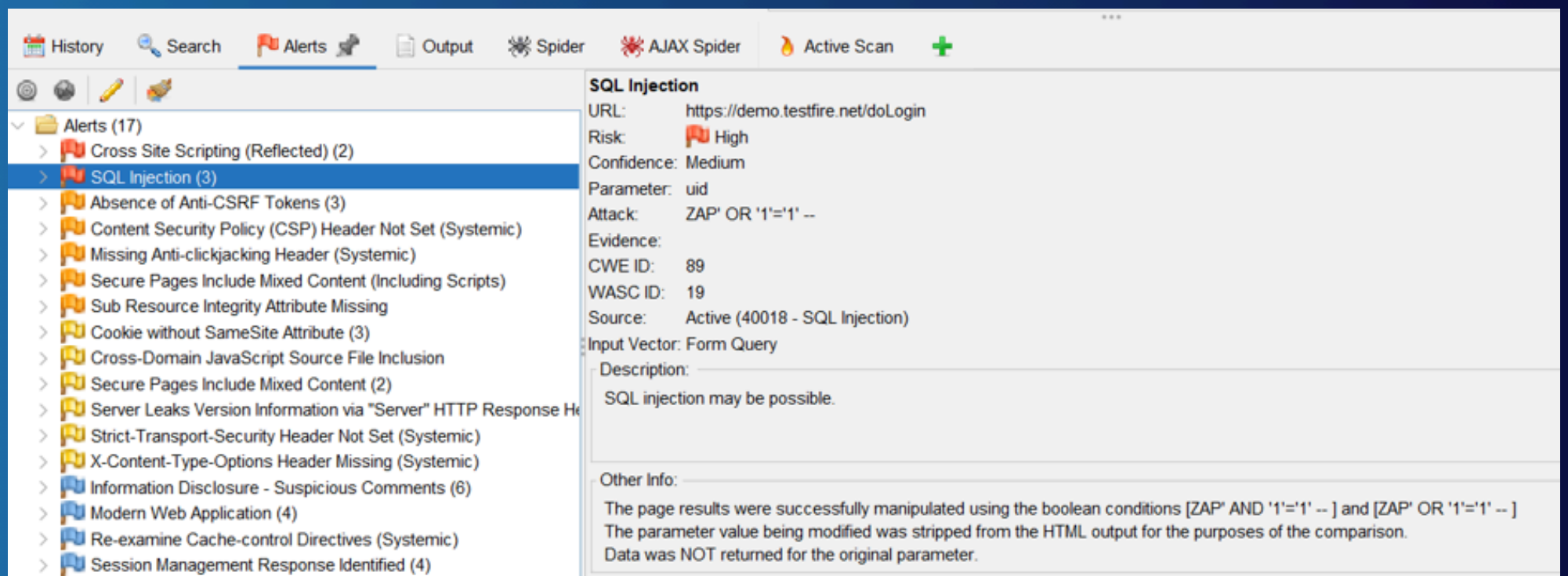
Payload Detected:

ZAP' OR '1'='1' --

Remediation

- Use parameterized queries (Prepared Statements)
 - Validate and sanitize all user inputs
 - Avoid dynamic SQL query construction
 - Implement least-privilege database access
- Follow OWASP SQL Injection Prevention Guidelines

Screenshot



Description:

SQL injection may be possible.

Other Info:

The page results were successfully manipulated using the boolean conditions [ZAP' AND '1'='1' --] and [ZAP' OR '1'='1' --]
The parameter value being modified was stripped from the HTML output for the purposes of the comparison.
Data was NOT returned for the original parameter.

Solution:

Do not trust client side input, even if there is client side validation in place.
In general, type check all data on the server side.
If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Observation

The application appears vulnerable to SQL Injection attacks through user-supplied input parameters. Proper server-side validation and secure database query practices should be implemented to mitigate this risk.

Risk Summary Table

Risk Level: High

CWE ID: 89

OWASP Category: Injection

FINDING #3

ABSENCE OF ANTI-CSRF TOKENS

Risk level

 Medium Risk

Description

Cross-Site Request Forgery (CSRF) is a vulnerability that forces authenticated users to perform unwanted actions on a web application without their knowledge. The assessment identified that Anti-CSRF tokens were not implemented in certain HTML forms, increasing the risk of unauthorized requests.

Impact

- Unauthorized actions may be performed on behalf of users
- User accounts may be manipulated without consent
 - Increased risk of session abuse
 - Reduced protection against forged requests

Evidence

OWASP ZAP detected the absence of Anti-CSRF tokens in the application's login form.

Affected URL:

<https://demo.testfire.net/login.jsp>

CWE ID: 352

Remediation

- Implement Anti-CSRF tokens in all sensitive forms
 - Validate CSRF tokens on the server side
- Use secure frameworks that provide built-in CSRF protection
 - Follow OWASP CSRF Prevention Guidelines

Screenshot



Source:	Passive (10202 - Absence of Anti-CSRF Tokens)
Input Vector:	
Description:	No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting
Other Info:	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 2: "btnSubmit" "passw" "uid"].
Solution:	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard.
Reference:	https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/352.html

Observation

The application does not implement Anti-CSRF protection mechanisms for certain forms. This may allow attackers to trick authenticated users into submitting unintended requests.

Risk Summary Table

Risk Level: Medium

CWE ID: 352

OWASP Category:

Broken Access Control /

CSRF

FINDING #4

CONTENT SECURITY POLICY (CSP) HEADER NOT SET

Risk level

 Medium Risk

Description

The website does not implement a Content Security Policy (CSP) header. CSP is an additional security layer that helps prevent attacks such as Cross-Site Scripting (XSS) and malicious code injection. Without CSP, browsers have fewer restrictions on which resources and scripts can be executed, increasing the attack surface of the application.

Impact

- Increased risk of Cross-Site Scripting (XSS) attacks
 - Malicious scripts may execute in users' browsers
- Reduced protection against content injection attacks
 - Increased exposure to client-side security threats

Evidence

OWASP ZAP detected that the Content-Security-Policy (CSP) header was not configured on the website.

Affected Resource:

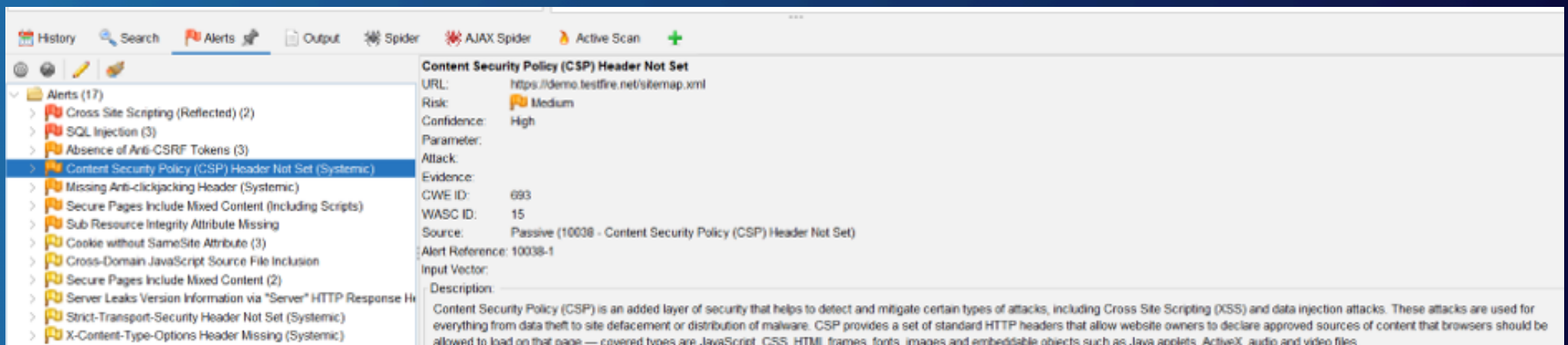
<https://demo.testfire.net>

CWE ID: 693

Remediation

- Configure the Content-Security-Policy HTTP header
- Restrict trusted sources for scripts, styles, images, and other resources
 - Allow content only from approved domains
- Follow OWASP Content Security Policy best practices

Screenshot



Solution: _____	
Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.	
Reference: _____	
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/	
Alert Tags: _____	
Key	Value
POLICY_PENTEST	
SYSTEMIC	https://www.zaproxy.org/docs/desktop/addons/common-library/alerttags/#systemic
CWE-693	https://cwe.mitre.org/data/definitions/693.html
OWASP_2017_A06	https://owasp.org/www-project-top-ten/2017/A6_2017-Security_Misconfiguration.html
OWASP_2025_A02	https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/

Observation

The absence of a CSP header reduces browser-level protection against malicious scripts and content injection attacks. Implementing CSP can significantly improve the security posture of the application.

Risk Summary Table

Risk Level: Medium

CWE ID: 693

OWASP Category: Security Misconfiguration

FINDING #5

MISSING ANTI-CLICKJACKING HEADER

Risk level

 Medium Risk

Description

Clickjacking is an attack technique in which a malicious website tricks users into clicking on hidden or disguised elements. The assessment identified that the application does not implement anti-clickjacking protection headers such as X-Frame-Options or the Content Security Policy (CSP) frame-ancestors directive.

Impact

- Users may be tricked into performing unintended actions
 - Sensitive operations may be executed without user awareness
- Increased risk of phishing and social engineering attacks
- Reduced protection against malicious framing of web pages

Evidence

OWASP ZAP detected that the Anti-Clickjacking protection header was not configured.

Affected URL:

<https://demo.testfire.net/login.jsp>

Missing Header:

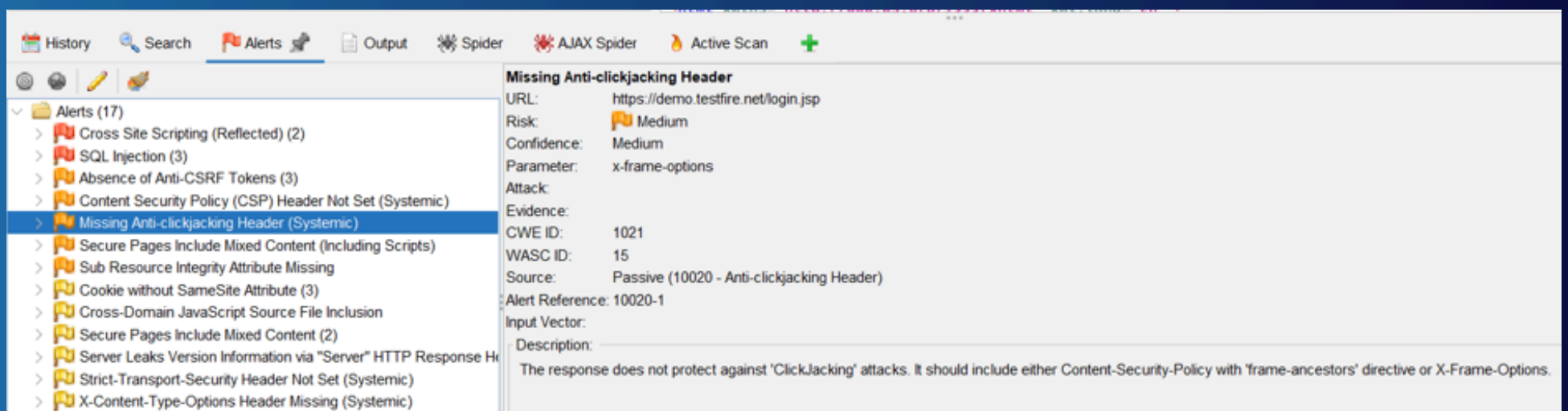
X-Frame-Options

CWE ID: 1021

Remediation

- Configure the X-Frame-Options header
- Use X-Frame-Options: DENY or SAMEORIGIN
- Implement the Content Security Policy (CSP) frame-ancestors directive
- Regularly review security headers across all web pages

Screenshot



Solution:
Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference:
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options

Observation

The application does not prevent its pages from being embedded within external websites. This increases the risk of clickjacking attacks and unauthorized user interactions.

Risk Summary Table

Risk Level: Medium
CWE ID: 1021

OWASP Category: Security Misconfiguration

SECURITY

RECOMMENDATIONS

1. Implement input validation and output encoding to prevent Cross-Site Scripting (XSS).
2. Use parameterized queries and prepared statements to eliminate SQL Injection vulnerabilities.
3. Implement Anti-CSRF tokens for all sensitive forms and transactions.
4. Configure Content Security Policy (CSP) headers to reduce client-side attack risks.
5. Enable Anti-Clickjacking protection using X-Frame-Options or CSP frame-ancestors directives.
6. Regularly perform vulnerability assessments and security testing.
7. Follow OWASP Top 10 security best practices during development and deployment.

CONCLUSION

The vulnerability assessment identified several High and Medium risk security issues within the target application. These findings demonstrate the importance of secure coding practices, proper security configurations, and continuous security testing.

By implementing the recommended remediation measures, the organization can significantly improve the security posture of the application and reduce the likelihood of successful cyber attacks.

Regular security assessments and adherence to OWASP best practices are strongly recommended to maintain a secure web application environment.